

CSC 505, Homework 4

Due date: Monday, November 11, before 9:00 PM

Please submit the homework **in printed form as a PDF file** before 9 PM on the due date. Late homework will be accepted only in circumstances that are grounds for excused absence under university policy. The university provides mechanisms for documenting such reasons (severe illness, death in the family, etc.). Arrangements for turning in late homework must be made by the day preceding the due date.

Write-up submission: Submit your write-up for the questions in HW4 in a single PDF file. Answers must be typed. Scans of handwritten answers are NOT acceptable. Start each answer on a new page. Clearly label the question number in bold. **Do not include your code for the programming part in the PDF.**

Go to Gradescope (NOT Moodle) at <https://www.gradescope.com/courses/865193> and choose the **HW4 Write-up** assignment. Once you upload your PDF, you will be shown the following screen:

Question Outline

Select a question or a page.

Title	Points
1 Q1	21.0 pts
2 Q2	18.0 pts
3 Q3	10.0 pts

The screenshot shows the Gradescope interface with three question cards. Card 1 (Q1) is selected and shows the question text and instructions. Card 2 (Q2) and Card 3 (Q3) are also visible, showing their respective questions. The interface includes a search bar and a 'C' icon for each card.

You need to match each of your answers to the PDF pages:

- Select a question on the left
- Select **ALL** pages that contain any part of its answer (even if that page has other answers too). You can zoom on the top right
- Repeat for each question
- Submit

If needed, you can change this mapping later by going back to your submission and clicking on “reselect pages”.

Code submission: Your program must be submitted as a single code file (*.java or *.py, no zips). For instructions on how to set up your testing environment, refer to the [HW4 Programming Assignment setup instructions](#) on our Moodle page. Go to Gradescope at <https://www>

[.gradescope.com/courses/865193](https://gradescope.com/courses/865193) and choose the **HW4 Programming** assignment and submit your single code file.

All assignments for this course are intended to be individual work. Turning in an assignment that is not your work is cheating. The Internet is not an allowed resource! Copying text, code, or other content from the Internet (or other sources) is plagiarism. Any tool/resource must be approved by the instructor and identified and acknowledged clearly in any work turned in; anything else is plagiarism.

If an academic integrity violation occurs, the offending student(s) will be assessed a penalty at least as severe as getting a 0 for the whole homework assignment for which the violation occurred. The case will be reported to the Office of Student Conduct.

General instructions: Please justify all answers. Answering with a single number, equation, or time/space complexity will reduce marks if not explicitly asked.

Instructions about describing an algorithm (taken from Erik Demaine): Be concise, correct, and complete. **To avoid deductions**, you provide (1) a textual description of the algorithm and, if helpful, flow charts and pseudocode; (2) at least one worked example or diagram to illustrate how your algorithm works; (3) argue the correctness of the algorithm; and (4) an analysis of the time complexity (and, if relevant, the space complexity) of the algorithm. Your overall goal is to communicate. If a grader cannot understand your solution, they cannot give you appropriate credit for it.

1. Purpose: Learn about Euler tours (12 points). An Euler tour of a strongly connected graph $G=(V, E)$ is a cycle that traverses each edge of G exactly once, although it may visit a vertex more than once.

a) Show that G has an Euler tour if and only if $\text{in-degree}(v) = \text{out-degree}(v)$ for each vertex $v \in V$ by showing:

- i) if a graph has an Euler tour, then $\text{in-degree}(v) = \text{out-degree}(v)$ for each vertex v .
- ii) if a strongly connected graph $G=(V, E)$ has $\text{in-degree}(v) = \text{out-degree}(v)$ at every vertex v in V , then G has an Euler tour.

b) Describe an $O(E)$ -time algorithm for finding an Euler tour in a strongly connected graph $G=(V, E)$ with $\text{in-degree}(v) = \text{out-degree}(v)$ at every vertex v in V of G . Hint: merge edge-disjoint cycles.

2. Purpose: Reinforce your understanding of MST algorithms and practice algorithm design (14 points). Let T be the Minimum Spanning Tree of a graph $G=(V, E, w)$. Suppose G is connected, $|E| \geq |V|$, and all edge weights are distinct. Denote T^* the MST of G and $ST(G)$ be the set of all spanning trees of G . A second-best MST is a spanning tree T such that $w(T) = \min\{w(T) : T \in ST(G) - \{T^*\}\}$.

a) (4 points) Show that T^* is unique but that the second-best MST T_2 need not be unique.

b) (6 points) Please modify Kruskal's algorithm to design an efficient algorithm to compute G 's second-best MST. Use the following edge-switch lemma: G contains an edge $(u,v) \in T^*$ and another edge $(s,t) \notin T^*$ such that $(T^* - \{(u,v)\}) \cup \{(s,t)\}$ is a second-best minimum spanning tree of G . You don't have to prove the edge-switch lemma.

3. *Purpose: Reinforce your understanding of Dijkstra's shortest path algorithm and practice algorithm design (16 points).* Suppose you have a weighted, undirected graph G with positive edge weights and a start vertex s .

a) (6 points) Describe a modification of Dijkstra's algorithm that runs (asymptotically) as fast as the original algorithm and assigns a binary value $usp[u]$ to every vertex u in G , so that $usp[u]=1$ if and only if there is **a unique shortest path from s to u** . In addition to your modification, be sure to provide arguments for both the correctness and time-bound of your algorithm and an example.

b) (10 points) Implement the algorithm described in a). As you can see in the Dijkstra code framework, `Dijkstra.py/java` has a method called `Dijkstra_alg`. The method has an input parameter list (n, e, mat, s) , where n = number of vertices of G , e = number of *undirected* edges of G , mat = an $e \times 3$ matrix that defines the edges of G , and s = the source vertex of Dijkstra's algorithm. **Assume the vertices of G are numbered $1 \dots n$.** In mat , each row consists of three integers $\langle u, v, weight \rangle$. Here, u and v define the edge (u,v) , and $weight$ is the corresponding edge weight. The method returns an $n \times 2$ matrix, where the i th row contains the path length and the usp value for the shortest path between source and vertex i for $i \in \{1, \dots, n\}$. Below are more detailed examples (referring to the first two test cases). You can create additional methods if required, but do not change existing methods' names, and any existing code - points will be cut if you do. To avoid loss of marks, you should use basic arrays to implement the priority queue in Dijkstra's algorithm and not use any packages. You can code this in either Java or Python. Another file, `DijkstraTest.py/java`, has been provided, which tests your code for various inputs. You need to test your code on VCL using this file (instructions for VCL setup and test can be referred from the file *HW4 Programming Assignment Setup Instructions* on our Moodle page). Please only submit the file `Dijkstra.py/java` and not the test file. We will check your code on VCL using the provided cases plus some (unknown) inputs. To avoid loss of marks, please ensure that all provided test cases pass on VCL by using the test file.

Some examples:

Testcase1

Input values

```
n = 5,  
e = 6,  
mat = [[1, 2, 9], [1, 3, 6], [1, 4, 5], [1, 5, 3], [3, 2, 2], [3, 4, 4]],  
s = 1
```

Output

Ans = [[0, 1], → Shortest path weight **source** to vertex 1 is 0, usp is 1
[8, 1], → Shortest path weight **source** to vertex 2 is 8, usp is 1
[6, 1], → Shortest path weight **source** to vertex 3 is 6, usp is 1
[5, 1], → Shortest path weight **source** to vertex 4 is 5, usp is 1
[3, 1]] → Shortest path weight **source** to vertex 5 is 3, usp is 1

All shortest paths originating from source vertex 1 are unique, and the corresponding usp value is 1.

Testcase2:

Input values

n = 5,
e = 5,
mat = [[1, 3, 2], [1, 5, 3], [2, 5, 3], [4, 1, 1], [4, 2, 1]]
s = 4

Output

Ans = [[1, 1], → Shortest path weight **source** to vertex 1 is 1, usp is 1
[1, 1], → Shortest path weight **source** to vertex 2 is 1, usp is 1
[3, 1], → Shortest path weight **source** to vertex 3 is 3, usp is 1
[0, 1], → Shortest path weight **source** to vertex 4 is 0, usp is 1
[4, 0]] → Shortest path weight **source** to vertex 5 is 4, usp is 0

There are two shortest paths of length 4 from source vertex 4 to vertex 5. Therefore, the usp value of the fifth row is 0. All other shortest paths originating from vertex 4 are unique; therefore, the corresponding usp value is 1.